

令和 7 年度卒業論文

音楽特徴量に基づく人間親和的なイメージ画像生成手法の提案

2026 年 1 月 12 日

愛知県立大学 情報科学部 情報科学科

知能メディアコース 何研究室

2022311042 神野友哉

目次

1 はじめに	6
1.1 研究の背景	6
1.2 研究の目的	6
1.3 本論文の構成	6
2 提案手法	7
2.1 基礎知識	7
2.2 提案手法の概要	7
2.3 提案手法の詳細	8
2.3.1 音楽データセット GTZAN	8
2.3.2 音響特徴量の抽出	8
2.3.3 単語ラベル生成モデル	9
2.3.4 機械学習の効率化	10
2.3.5 画像生成用プロンプトの生成	11
3 実装	13
3.1 実験環境	13
3.2 実験方法	13
3.3 実験結果	14
3.3.1 音楽と生成画像の雰囲気の合致アンケート結果 . . .	14
4 考察	17
5 おわりに	18
謝辞	19
参考文献	20
付録	21
A ラベル付与および特徴量保存処理	23
B CSV 形式データセットの定義	28
C マルチラベル分類ニューラルネットワーク	29

D 半教師あり学習の学習ループ	30
E 推論および単語ラベル抽出処理	32
F 外部音源を用いた評価例	33
G 特徴抽出プログラム	34
H GTZAN データセットの読み込み方法	37

図 目 次

図 1 : 単語ラベル生成モデルの構成図	10
図 2 : 音楽から生成された単語ラベルに基づき生成された画像 例. (a) プロンプト A から生成された画像, (b) プロンプ ト B から生成された画像	12
図 3 : アンケートで提示した生成画像. (a) 一曲目 Q1 から生成 された画像, (b) 二曲目 Q2 から生成された画像, (c) 三 曲目 Q3 から生成された画像, (d) 四曲目 Q4 から生成さ れた画像, (e) 五曲目 Q5 から生成された画像	15
図 4 : アンケート結果の集計グラフ	16

表 目 次

表 1： 回答者間の類似度行列	16
---------------------------	----

1 はじめに

1.1 研究の背景

音楽が聴き手に示す情報の大半は”音”と”名前”に限られているため、記憶の中の”音”の情報だけを頼りに、思い出したい音楽を探すと大変苦労した経験がある。そのため、”視覚”情報を追加できると印象を深められると考えた。音楽をジャンルとしての単語に分類する研究は盛んに行われてきたが [2]-[4], 音楽を中間材料としての単語に対応させてから画像生成を試みるという研究は、確認する限り全く無いものであった。

1.2 研究の目的

本研究では、音楽の持つ情報を機械的に画像に対応させる方法を検討し、人間的な感性を伴った対応が実現可能かどうか確かめることが目的である。より具体的には、音楽から単語への変換及び単語から画像生成のプロセスにおける有効性があるか、また、音楽と生成画像の雰囲気がどの程度合致しているか検証する。

1.3 本論文の構成

本論文では、音楽から人間にとって適切だと思えるような画像を生成する方法について研究を行う。本論文の以下の構成は次のようになっている。第2章では、音楽から中間表現としてのテキスト情報の対応及び外部の画像生成サイトを用いて生成する方法を提案する。第3章では、実際に動かしてアルゴリズムの研究を実施する。第4章では、提案されたアルゴリズムの評価を行う。最後に、第5章で本論文の結論を述べる。なお、付録として 実行環境 Google Colaboratory での python プログラムを加えた。

2 提案手法

2.1 基礎知識

特徴量： データの特徴を数値化して、機械学習などの分析で扱えるようにしたもの

ニューラルネットワーク (以降 NN と称する)： 人間の脳の神経細胞 (ニューロン) の働きを模したコンピューターの数理モデル

教師あり学習： 正解ラベル付きのデータセット (入力 feature と出力 label の対応がある) を用いて AI モデルを学習させる機械学習の手法

半教師あり学習： 少量のラベル付きデータと、大量のラベルなしデータの両方を使用して機械学習モデルを訓練する手法

活性化関数： NN の各ニューロンにおいて、入力値から出力値に変換する関数

シグモイド関数： いかなる実数入力であっても 0 から 1 の間の値に変換する関数であり、機械学習の二値分類で確率を出力したい際に用いられる

GTZAN： 音楽 10 ジャンルから構成され、各ジャンルにつき 100 曲、合計 1000 曲、1 曲 30 秒、の無料で活用可能な学術用音楽データセット [6].

2.2 提案手法の概要

音楽は聴覚を通じて人に感情や雰囲気を想起させる表現媒体であるが、その印象は数値や明確な言語として表現することが難しい。一方で、近年発展が著しい画像生成 AI は、主にテキスト情報を入力としており、音楽のような非言語的、感覚的情報を直接扱うことは困難である。また、音楽をジャンルや感情語に分類する研究は数多く存在するものの、音楽の雰囲気を中間的な単語表現に変換し、その単語を用いて画像生成を行う手法については十分に検討されていない。さらに、音楽の受け取り方には個人差が存在するにもかかわらず、個人の聴感特性を考慮した音楽と視覚表現の対応に関する研究は少ない。そこで本研究では、音楽から抽出される特徴

量を基に雰囲気を表す単語を生成し、その単語を用いて画像を生成することが可能かどうか。また、その結果が人間の主観的印象とどの程度一致するか、聴感特性に個人差がどの程度あるかを明らかにすることを課題とする。

2.3 提案手法の詳細

本研究では、音楽から視覚的イメージを生成するために、音楽データを数値的特徴量へ変換し、その特徴量から音楽の雰囲気を表す単語ラベルを生成する手法を用いた。本節では、使用した音楽データセット、特徴量抽出方法、および単語生成モデルについて説明する。

2.3.1 音楽データセット GTZAN

本研究では、音楽ジャンル分類のベンチマークとして広く用いられている GTZAN データセットを使用した [6]。本データセットは Google Colaboratory 上での実験を想定し、ローカル環境ではなく Google Drive 上のクラウドストレージに保存した状態で読み込んでいる。これは、実験環境の再現性を確保し、大容量データを扱う際の利便性を考慮したためである。

音声データの読み込みには Python ライブラリである `librosa` を使用した。各音声ファイルからは、楽曲冒頭の 5 秒間のみを切り出して利用した。サンプリング周波数は 22,050 Hz とし、入力長を統一するため、 $22,050 \times 5 = 110,250$ サンプルを上限として波形データを取得した。これにより、楽曲長の違いによる影響を抑え、モデルへの入力条件を統一している。

対象ジャンルは GTZAN に含まれる 10 ジャンル (blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock) とし、各ジャンルにつき最大 100 曲を読み込んだ。なお、読み込み時にエラーが発生した音声ファイルについては、処理を中断せずにスキップする実装とした。GTZAN データセットの読み込みには、付録 H に示す読み込みプログラム（ソースコード 9）を用いた。

2.3.2 音響特徴量の抽出

音楽の雰囲気を数値的に表現するため、各楽曲から以下の音響特徴量を抽出した。

Zero Crossing Rate は波形がゼロを跨ぐ頻度を表し、音の高さや速さに関係する特徴量である。

Spectral Centroid は周波数軸上の重心を示し、音の明るさやバランスを表現する指標である。

Root Mean Square (RMS) は信号の二乗平均平方根であり、経時的な音の大きさを表す。

Spectral Bandwidth は周波数成分の広がりを示し、音の豪華さや厚みを反映する。

Tempo は 1 分間あたりのビート数を表し、楽曲の速さを示す指標である [1]。

Spectral Flatness はスペクトルの平坦さを示し、音がノイズ的か単音的かを判別する特徴量である [1]。

Spectral Contrast は周波数帯域におけるピークと谷の差を表し、音の鋭さや明瞭さを示す。

Mel-Frequency Cepstral Coefficients (MFCC) は、人間の聴覚特性を考慮した周波数尺度に基づく特徴量であり、音楽信号の特徴を表現するために広く用いられている。

本研究では、Spectral Contrast を 7 次元、MFCC の平均値および標準偏差を合計 26 次元、その他の特徴量を 6 次元として抽出し、合計 39 次元の音響特徴量を得た。音響特徴量の抽出には、付録 G に示す特徴抽出プログラム（ソースコード 8）を用いた。

2.3.3 単語ラベル生成モデル

抽出した 39 次元の音響特徴量を入力として、音楽の雰囲気を表す単語ラベルを生成するモデルを構築した。本研究では、浅層ニューラルネットワークを用い、出力層にシグモイド関数を適用することで、複数の単語ラベルを同時に出力可能なマルチラベル分類を実現した。シグモイド関数を用いることで、各単語ラベルが付与される確率を独立に推定でき、音楽が複数の雰囲気を同時に持つ場合にも対応可能となる。モデルの構成を図 1 に示す、生成される単語ラベルは以下のカテゴリに分類される。

- 形容詞（音の印象）：明るい、暗く寂しい、静か・滑らか、激しい・力強い、幻想的・夢想的、高貴・荘厳、現代的、情熱的
- 名詞（音から想起される情景）：自然風景、都市・人工物、時間・空模様、抽象・感情、神秘的な場所

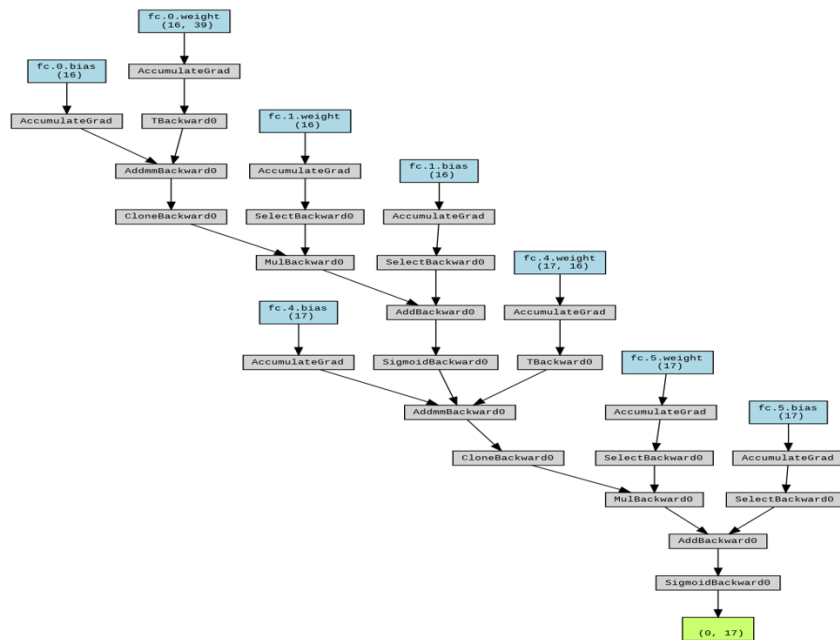


図 1: 単語ラベル生成モデルの構成図

- 動作表現：鳴り響くように
- 季節：春，夏，秋，冬
- 色：赤，橙，黄，黄緑，緑，水色，青，紫，ピンク，茶色，黒，白

生成された単語ラベルは、画像生成を行う生成 AI に入力され、音楽の雰囲気に基づいた視覚的イメージの生成に利用される。

2.3.4 機械学習の効率化

本研究では、単語ラベル付き音楽データの収集コストを抑えつつ、学習に用いるデータ量を確保するため、半教師あり学習を用いて学習データの水増しを行った。

一般に、音楽の雰囲気を表す単語ラベルの付与には人手による主観的判断が必要となり、大量のラベル付きデータを用意することは困難である。そこで本研究では、少量のラベル付きデータと、大量のラベルなしデータを併用する半教師あり学習を採用した。

具体的には、GTZAN データセットに含まれる 10 ジャンルの楽曲のうち、各ジャンル 5 曲、合計 50 曲をラベル付き学習データとして用いた。こ

これらの楽曲に対しては、筆者が楽曲を聴取し、音楽の雰囲気に基づいて単語ラベルを付与した。一方、残りの 950 曲についてはラベルを付与せず、ラベルなし学習データとして利用した。

学習の流れとしては、まずラベル付きデータのみを用いて単語ラベル生成モデルの初期学習を行った。次に、学習済みモデルを用いてラベルなしデータに対する単語ラベルの推定を行い、その推定結果を擬似ラベルとして付与した。この擬似ラベル付きデータを、元のラベル付きデータと合わせて再学習することで、実質的に学習データ数を増加させた。

このような半教師あり学習の導入により、少量の人手ラベルから得られた知識を、大量のラベルなし音楽データへ拡張することが可能となり、モデルの汎化性能向上および学習効率の改善が期待される [5]。

なお、モデルの評価には、GTZAN データセットには含まれないフリー音楽データ 10 曲を用いた。これらのテスト楽曲については、筆者が便宜上、音楽と生成された単語ラベルおよび画像との対応関係を確認し、提案手法が未知の楽曲に対しても一定の妥当性を持つかを定性的に検証した。

2.3.5 画像生成用プロンプトの生成

前節で生成された単語ラベルは、そのまま画像生成 AI に入力するのではなく、画像生成に適した形式へと整理した。本研究では、単語ラベルを意味的なカテゴリごとに分類し、各カテゴリ内で出力確率の高い単語を選択する方法を用いた。具体的には、形容詞カテゴリから上位 2 語、情景を表す名詞カテゴリから上位 2 語、色カテゴリから上位 1 語、季節カテゴリから上位 1 語を選択した。これは、名詞: 構図, 形容詞: 雰囲気, 色: 視覚的統一感, 季節: 時間的文脈として十分な情報を含有すると考察したためである。そうして選択された単語は、確率の高い順に連結し、画像生成 AI に入力するテキストプロンプトとして使用した。このような選択規則により、プロンプトが過度に冗長になることを防ぎつつ、音楽の雰囲気を多面的に表現することが可能となった。さらに、画像生成 AI に対しては、単語列挙型のプロンプトとして解釈されるよう、あらかじめ共通の指示文（プロンプト命令）を付与した。

以降、単語を複数列挙するのでそれで絵を描いてもらいます。

各々の絵はそれまでの絵との関連はないようにしたい

この指示文では、以降に列挙される単語群のみを手掛かりとして 1 枚の画像を生成すること、および各画像生成がそれ以前の生成結果と意味的な

関連を持たないようにすることを明示した. 実際のプロンプトはこの指示文に続けて, 前述の規則により選択された単語ラベルを空白区切りで記述する形式とした. 以下に, 生成に用いたプロンプトの一例を示す.

プロンプト A (後述する楽曲 Q1 の生成プロンプト)

暗く寂しい 冬 神秘的な場所 自然風景 青 幻想的・夢想的

上記プロンプト A を入力して生成された画像を, 図 2(a) に示す.

プロンプト B (後述する楽曲 Q2 の生成プロンプト)

都市・人工物 高貴・荘厳 静か・滑らか 秋 時間・空模様 茶色

同様に, プロンプト B を入力して生成された画像を, 図 2(b) に示す.



(a)



(b)

図 2: 音楽から生成された単語ラベルに基づき生成された画像例. (a) プロンプト A から生成された画像, (b) プロンプト B から生成された画像

このように, プロンプトの文法構造を固定し, 単語ラベルのみを変化させることで, 生成される画像の差異が音楽から生成された単語に起因するよう配慮した.

3 実装

3.1 実験環境

本研究における実験環境は以下のとおりである.

- PC : 自宅デスクトップ PC
- プロセッサ : Intel(R) Core(TM) i7-9700K CPU @ 3.60GHz
- GPU : NVIDIA GeForce RTX 2070 (8GB)
- OS : Windows 11 Home
- RAM : 16GB
- 開発・実験場所 : 自宅

音響特徴量の抽出およびニューラルネットワークの学習・評価, 画像生成プロンプトの生成に Google Colaboratory を利用した. 上記の処理においては, ブラウザ上のクラウドで活用可能な環境であるため, 利用した媒体や場所によって, 実験内容に何らかの影響を及ぼすことはない.

3.2 実験方法

本研究では, 音楽から生成された画像が, 元の音楽の雰囲気とどの程度対応しているかを検証するため, 以下の手順で実験を行った.

GTZAN データセット [6] をダウンロードし からダウンロードし, Google Drive 上に保存する. 音響特徴量の抽出には, 付録 G (ソースコード 8) に示す特徴抽出プログラムを用い, 各楽曲の冒頭 5 秒間の音声波形から 39 次元の音響特徴量を算出し, csv ファイルに保存した.

次に, 単語ラベル付き学習データの作成を行った. GTZAN データセットに含まれる 10 ジャンルから, 各ジャンル 5 曲, 合計 50 曲を選択し, 付録 A (ソースコード 2) に示す楽曲再生およびラベル付与プログラムを用いて, 筆者が楽曲を聴取しながら, 音楽の雰囲気を表す単語ラベルを主観的に付与した. これらのデータをラベル付きデータとして用いた.

一方, 残りの楽曲についてはラベルを付与せず, 音響特徴量のみを抽出したデータをラベルなしデータとした. ラベル付きデータおよびラベル

なしデータは、同一の CSV ファイルに保存され、付録 B（ソースコード 3）に示す CSVDataset クラスを用いて、条件分岐によりそれぞれ取得される。

次に、半教師あり学習による単語ラベル生成モデルの学習を行った。モデルには付録 C（ソースコード 4）に示すマルチラベル分類ニューラルネットワークを用い、ラベル付きデータによる教師あり学習と、ラベルなしデータに対する擬似ラベル学習を同一エポック内で交互に実行した。学習ループの詳細は付録 D（ソースコード 5）に示す。学習は 1000 エポック実行し、損失関数にはバイナリクロスエントロピーを用いた。

学習済みモデルを用いて、各楽曲に対する単語ラベルの出現確率を推定し、付録 E（ソースコード 6）に示す推論および単語抽出処理により、カテゴリごとに確率の高い単語を選択した。選択された単語群を空白区切りで連結し、画像生成用プロンプトとして整形した。

その後、整形されたプロンプトを外部の画像生成 AI に入力し、音楽の雰囲気に基づく画像を生成した。画像生成時には、プロンプト構造を固定し、単語ラベルのみを変化させることで、生成結果の差異が音楽に由来するように配慮した。

最後に、音楽と生成画像の対応関係を評価するため、主観評価アンケートを実施した。被験者には音楽と対応する生成画像を提示し、両者の雰囲気の合致度について、0 から 10 の 11 段階尺度で評価を求めた。アンケート結果を集計し、平均値および回答者間の相関係数を算出することで、音楽と画像の雰囲気対応の傾向および個人差を分析した。

3.3 実験結果

3.3.1 音楽と生成画像の雰囲気の合致アンケート結果

本アンケートは、音楽と生成画像の雰囲気の合致度について評価することを目的として実施した。評価は 0 から 10 の 11 段階尺度を用い、0 を「雰囲気が全く異なる」、10 を「雰囲気が完全に一致している」と定義した。被験者数は $N=5$ とし、回答は匿名で収集した。

アンケートの設問数は 5 つであり、各設問はフリー素材楽曲の url と 0-10 のラジオボックスのペアで構成されている。使用した曲の雰囲気とその url、提示した生成画像について以下に示す。

- 一曲目 Q1：お化け屋敷のような暗い空間に居そうな印象でオカ

ルト寄り. リズムが時計の音のようである. <https://www.dova-s.jp//bgm/play22185.html>

- 二曲目 Q2 : 和風な曲と言えば最も近いが, どこか民族的な音楽としても感じられる. 伝統的な楽器と三味線のような弦楽器が印象的. <https://dova-s.jp/bgmm/play22894.html>
- 三曲目 Q3 : 全体的に落ち着いた曲であり, 雨が降っていそうな印象を受ける. 短調だが暗すぎない. リズムが突発的な感覚で刺激を含む. <https://www.dova-s.jp/bgmm/play22919.html>
- 四曲目 Q4 : リズムが豪快であり, 音の和音が乱れていて不快感を抱く. とにかく壮大で, しかし重めな印象がある元気そうな曲である. <https://www.dova-s.jp//bgmm/play22899.html>
- 五曲目 Q5 : 未来感を感じる曲であり, 電子音が声のように入れ込まれていて聞き飽きない. <https://dova-s.jp/bgmm/play22903.html>

上記の楽曲 Q1 ~ Q5 はそれぞれ図 3(a)~図 3(e) に対応する.



図 3: アンケートで提示した生成画像. (a) 一曲目 Q1 から生成された画像, (b) 二曲目 Q2 から生成された画像, (c) 三曲目 Q3 から生成された画像, (d) 四曲目 Q4 から生成された画像, (e) 五曲目 Q5 から生成された画像

音楽と生成画像の雰囲気の一緻度に関するアンケート結果を図 4 に示す.

図 4 より, 回答者平均が中立値である 5 を約 1.2 ポイント下回る設問 (Q2, Q5), 中立値とほぼ一致する設問 (Q3), および中立値を約 1.2 から 2.4 ポイント上回る設問 (Q1, Q4) が確認された. このことから, 音楽

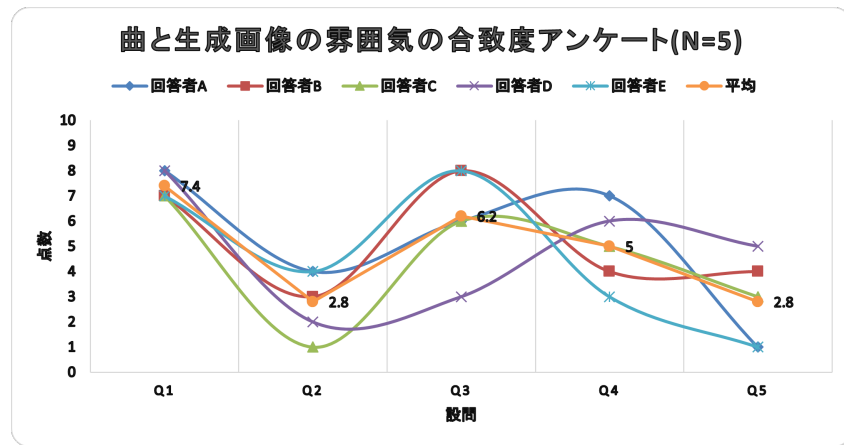


図 4: アンケート結果の集計グラフ

と生成画像の雰囲気の合致度には設問間でばらつきが存在することが分かる。

また、表 1 に、図 4 の各回答者間の評価結果の相関行列を示す。

表 1: 回答者間の類似度行列

	回答者 A	回答者 B	回答者 C	回答者 D	回答者 E
回答者 A	1				
回答者 B	0.5319	1			
回答者 C	0.7332	0.8427	1		
回答者 D	0.4604	0.2512	0.6696	1	
回答者 E	0.7005	0.8566	0.6414	0.0218	1

表 1 より、回答者 B と回答者 C、回答者 B と回答者 E の間で高い、音楽と生成画像の雰囲気の合致度の感じ方の類似が確認できる一方、回答者 D と回答者 E の間では極めて低く、被験者間で個人差が存在することが示唆される。しかし、相関係数は全て正の値であったため、音楽と生成画像間の共通情報の取得の類似性を認められると確認した。

4 考察

本研究により、音楽から抽出した音響特徴量を基に単語ラベルを生成し、それを用いて画像生成 AI による視覚的イメージの生成が可能であることが確認された、また、アンケート結果から、一部の楽曲においては、音楽と生成画像の雰囲気がある程度一致していると評価される例も見られ、本手法が音楽と視覚表現を結び付ける手段として一定の有効性を持つことが示唆された。一方で、評価結果全体を見ると、設問間で合致度にばらつきが存在し、必ずしも安定して高い一致度が得られたとは言い難い。この要因として、いくつかの課題が考えられる。

第一に、学習に用いたデータセット数および生成ラベル数の不足が挙げられる。GTZAN データセットはジャンル分類用途としては広く利用されているものの、音楽の雰囲気や情景を表す単語生成を目的とした場合には、必ずしも十分な多様性を持つとは限らない可能性がある。また、単語ラベルの種類や数が限定的であったため、音楽が持つ複雑な印象を十分に表現しきれなかったと考えられる。

第二に、学習データセット作成および単語ラベル設定における属人性の影響が考えられる。本研究では、単語ラベルの定義や意味付けを筆者の主観に基づいて行っており、これがモデルの学習結果や生成画像に反映された可能性がある。アンケートにおいて被験者間で評価の相関に差が見られた点も、音楽の捉え方や視覚的イメージの想起に個人差が存在することを示している。

第三に、音楽と画像という異なる情報媒体間の本質的な差異も、合致度が十分に高まらなかった要因の一つと考えられる。音楽は時間的に変化する聴覚情報であるのに対し、画像は空間的に固定された視覚情報であり、両者の情報量や表現内容は大きく異なる。このため、音楽の印象を単語へ変換し、さらに画像へ対応付ける過程において、情報の欠落や解釈のずれが生じやすかったと推察される。

以上より、本研究で提案した手法は、音楽から画像生成を行う試みとして一定の可能性を示した一方で、データセットの拡充、ラベル設計の客観化、および個人差を考慮したモデル設計など、今後解決すべき課題が多く残されていることが明らかとなった。

5 おわりに

本研究では、個人の聴感特性に基づいて音楽を画像に変換する手法について研究した。具体的には、音楽の離散的信号列から音響特徴量を抽出し、それを基に音楽の雰囲気を表す単語ラベルを生成するモデルを構築した。さらに、生成された単語ラベルを画像生成 AI の入力プロンプトとして用いることで、音楽の印象に基づいた視覚的イメージを生成する手法を提案した。

評価実験として、音楽と生成画像の雰囲気の合致度に関する主観評価アンケートを実施した結果、一部の楽曲においては音楽と生成画像の雰囲気がある程度一致していると評価される例が確認された。一方で、設問間および被験者間で評価結果にばらつきが見られ、音楽と生成画像の対応関係の捉え方には個人差が存在することが示唆された。しかし、回答者間の相関行列においては全て正の相関係数が得られたことから、雰囲気の合致について、ある一定の共通した印象が共有されている可能性も示された。

以上より、本研究で提案した手法は、音楽という聴覚情報を中間的な単語表現を介して視覚情報へと変換する一つの枠組みを示した点において、一定の有効性と可能性を有すると考えられる。その一方で、学習データセットおよび単語ラベル設計の制約、ならびに音楽と画像という異なる情報媒体間の本質的な差異等々により、合致度が十分に高まらなかった可能性が考察された。

今後の課題としては、まず、より多様な音楽データおよび単語ラベルを用いた学習データセットの拡充、単語ラベル定義の客観化、および個人の聴感特性をより詳細にモデル化する手法の検討が挙げられる。これらの改善により、音楽と視覚表現の対応関係をより人間の感性に近い形で表現可能な、音楽画像生成手法の実現を期待したい。

謝辞

本論文の執筆にあたり、終始丁寧にご指導してくださった愛知県立大学
何立風 先生に感謝申し上げます。また調査にご協力いただきました研究
室及び、同学年皆様にお礼申し上げます。

参考文献

- [1] 中山 達喜, 吉田 真一 : 音楽分類における特徴量の検討,
https://www.jstage.jst.go.jp/article/fss/26/0/26_0_290/_article/-char/ja/ (2010).
- [2] Xiaoli Chu : Feature Extraction and Intelligent Text Generation of Digital Music, <https://pubmed.ncbi.nlm.nih.gov/35845909/> (2020).
- [3] G. Tzanetakis; P. Cook : Musical genre classification of audio signals, <https://ieeexplore.ieee.org/abstract/document/1021072> (2002).
- [4] Meyers, Owen Craigie : A mood-based music classification and exploration system, <https://dspace.mit.edu/handle/1721.1/39337> (2007).
- [5] Ledge.ai 編集部 : 教師なし学習とは — 教師あり学習や強化学習との違い・活用事例・代表的なアルゴリズムを紹介, <https://ledge.ai/articles/unsupervised> (2020).
- [6] GTZAN dataset: Music Genre Classification, <https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification> (2025 年 5 月アクセス) .

付録

前提実行

ソースコード 1: 実行必須

```
!pip install datasets
!pip install pydub
# !pip install torchviz graphviz # モデルを図示する用, 遅い

from google.colab import drive
import datasets
import numpy as np
import matplotlib.pyplot as plt
import librosa
from IPython.display import Audio
import warnings
warnings.filterwarnings('ignore')
import os
import time
from pydub import AudioSegment
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import pandas as pd

drive.mount('/content/drive')

DURATION = 2
SAMPLING_RATE = 22050
n_samples = int(DURATION * SAMPLING_RATE)
START_SAMPLE_INDEX = 0 # 0 で 0秒目、sampling_rate で 1秒目
START_SECOND = START_SAMPLE_INDEX / float(SAMPLING_RATE)
MUSIC_SAMPLES = len(waveforms)
# 音楽の速さは波形のゼロクロス数の大小と同義
ZERO_CROSSING_RATE_RELAX_BORDER = 0.05
ZERO_CROSSING_RATE_FAST_BORDER = 0.1
# 音楽の明るさは周波数スペクトル (横軸周波数、縦軸強さ。音の高さ低さ)の
# 重心中心と相関がある
SPECTRAL_CENTROID_DARK_BORDER = 1500
```

```

SPECTRAL_CENTROID_BRIGHT_BORDER = 3000
# ラウドネス (音の大きさ)、その二乗平均で人間の音の感じ方に近いらしい。
RMS_BORDER = 0.02
# スペクトル帯域幅 (音の地味さ豪華さ) 2500↑豪華、1500hz ↓地味
SPECTRAL_BANDWIDTH_RICH_BORDER = 2500
SPECTRAL_BANDWIDTH_SIMPLE_BORDER = 1500
# 保存ファイル名
csv_path = "/content/drive/MyDrive/卒研/mtrain_data.csv"
csv_ssl_path = "/content/drive/MyDrive/卒研/mtrain_data_ssl.csv"
model_save_path = "/content/drive/MyDrive/卒研/モデル/
    r7_05_01_mtrain_model.pth"
model_load_path = "/content/drive/MyDrive/卒研/モデル/
    r7_05_01_mtrain_model.pth"
model_ssl_save_path = "/content/drive/MyDrive/卒研/モデル/
    r7_05_21_mtrain_model_ssl.pth"
model_ssl_load_path = "/content/drive/MyDrive/卒研/モデル/
    r7_05_21_mtrain_model_ssl.pth"
# マルチラベルクラスタリングのシグモイド閾値
SIGMOID_THRESFOLD = 0.6

# ジャンルを配列で管理することで関数の取り回しを良くする
# jazz だけ何故か 99曲しかない
GENRE_LIST = ["blues", "classical", "country", "disco", "hiphop",
    , "jazz", "metal", "pop", "reggae", "rock"]
music_descriptive_features = [
    "zcr", "spec_centroid", "rms", "spec_bw",
    "tempo", "spec_flatness"
] + [f"spec_contrast{i}" for i in range(7)] + [f"mfcc{i}_mean"
    for i in range(13)] + [f"mfcc{i}_std" for i in range(13)]
# 音の印象カテゴリ (形容詞ベース)
Adjectives = ["明るい", "暗く寂しい", "静か・滑らか", "激しい・力強
    い", "幻想的・夢想的", "高貴・荘厳", "現代的", "情熱的"]
# 音から想起される情景 (名詞ベース)
Norns = ["自然風景", "都市・人工物", "時間・空模様", "抽象・感情
    ", "神秘的な場所"]
# 四季
Seasons = ["春", "夏", "秋", "冬"]
# 色
Colors = ["赤", "橙", "黄", "黄緑", "緑", "水色", "青", "紫", "ピ
    ンク", "茶色", "黒", "白"]
music_descriptive_labels = [] + Adjectives + Norns + [
    # 音の動作 (動詞ベース)
    "鳴り響くように" # 鳴り響く、包み込む、ささやく、弾ける

```

A ラベル付与および特徴量保存処理

本研究で用いたラベル付き学習データは、楽曲の一部を再生しながら、筆者が主観的に単語ラベルを付与することで作成した。付録 A では、その際に使用したプログラムを示す。

ソースコード 2: 楽曲再生およびラベル付与処理

```
# 入力 : ジャンル (GENRE_LISTのいずれか、でないならクラシックに)、番号 (1-100、ジャズは 1-99)
# 出力 : 0-998のwaveforms 配列上の添え字に
def genre_order_2_music_index(genre="classical", order=1):

    if genre not in GENRE_LIST:
        print(f"[用法] genreは{str(GENRE_LIST)}から選択しなさい^^f0^^9f^^a6^^8a")
        print("[デバッグ] genre_order_2_music_index_異常終了^^f0^^9f^^a6^^8a")
        return -1

    if order < 1 or order > 100 or (genre == "jazz" and (order < 1 or order > 99)):
        print(f"[用法] orderは1~100の範囲で指定しなさい^^f0^^9f^^a6^^8a。ただし jazz は1~99ね^^f0^^9f^^a6^^8a")
        print(f"[デバッグ] genre_order_2_music_index_異常終了^^f0^^9f^^a6^^8a")
        return -1

    # ジャンルと番号で配列上の位置を計算する
    music_index = GENRE_LIST.index(genre) * 100 + (order - 1)
    if genre not in ["blues", "classical", "country", "disco", "hiphop", "jazz"]:
        music_index = GENRE_LIST.index(genre) * 100 + (order - 2)
    return music_index

# 入力 : 0-998のwaveforms 配列上の添え字を
# 出力 : ジャンルと番号に
def music_index_2_genre_order(music_index=0):
    if music_index < 0 or music_index > 999:
        print(f"[用法] music_indexは0~998の範囲で指定しなさい^^f0^^9f^^a6^^8a")
```

```

        print("[デバッグ]_music_index_2_genre_order_異常終了^^f0^^9
              f^^a6^^8a")
        return "", -1
    if music_index <= 598:
        genre = GENRE_LIST[music_index // 100]
        order = music_index % 100 + 1
        return genre, order
    else:
        genre = GENRE_LIST[(music_index + 1) // 100]
        order = music_index % 100 + 2
        return genre, order

# gtzan サウンドを再生する部分だけ分離した
def gtzan_music_sound_by_index(music_index=0, sr=22050):
    display(Audio(waveforms[music_index][START_SAMPLE_INDEX:
        n_samples], rate=sr))

# スペクトルの配列から再生する
def waveform_sound(waveform, sr=22050):
    display(Audio(waveform, rate=sr))

# 入力 : 1ジャンル単位で指定
# 出力 : よく表す (仮)名詞形容詞文字列を生成、音楽分析箇所を再生する
def music_genre_test(genre="classical"):

    if genre not in GENRE_LIST:
        print(f"[用法]_{str(GENRE_LIST)}_から選択しなさい^^f0^^9f^^
              a6^^8a")
        print("[デバッグ]_music_genre_test_異常終了^^f0^^9f^^a6^^8a
              ")
        return -1

    first_index = GENRE_LIST.index(genre) * 100
    last_index = first_index + 100
    margin_after_jazz = 0

    if genre == "jazz":
        last_index = first_index + 99
    if genre not in ["blues", "classical", "country", "disco", "
        hiphop", "jazz"]:
        first_index = GENRE_LIST.index(genre) * 100 - 1
        last_index = first_index + 100
        margin_after_jazz = 1

```

```

for i in range(first_index, last_index):
    # 特徴抽出して説明 (仮)する
    cf = calculate_features(waveforms[i][START_SAMPLE_INDEX:
                                     n_samples], sr=22050)
    desc = describe_features(cf)

    print("特徴表現:", desc)
    print("^^f0^^9f^^8e^^b5□" + genre + str((i +
        margin_after_jazz) % 100 + 1) + "曲目の
        " + str(START_SECOND) + "秒目から
        " + str(DURATION) + "秒間を再生します!")
    gtzan_music_sound_by_index(music_index=i, sr=
        SAMPLING_RATE)

# 入力 : 1ジャンルの番号単位で指定
# 出力 : よく表す (仮)名詞形容詞文字列を生成、音楽分析箇所を再生する
def music_genre_order_test(genre="classical", order=1,
    save_feature_label_2_csv=False, csv_path=csv_ssl_path,
    enable_override=False):

    music_index = genre_order_2_music_index(genre, order)

    # 特徴抽出して説明 (仮)する
    cf = calculate_features(waveforms[music_index][
        START_SAMPLE_INDEX:n_samples], sr=22050)
    desc = describe_features(cf)

    print("特徴表現:", desc)
    print("^^f0^^9f^^8e^^b5□" + genre + str(order) + "曲目の
        " + str(START_SECOND) + "秒目から" + str(DURATION) + "秒間
        を再生します!")

    gtzan_music_sound_by_index(music_index=music_index, sr=
        SAMPLING_RATE)

# ちょっと時間差を持たせる。これがないと下のInputが出ない
time.sleep(0.5)

# 保存モードなら保存処理を行う
if save_feature_label_2_csv:
    # 保存先ファイルの存在確認
    if not os.path.exists(csv_path):
        print("[エラー]保存先のフォルダが存在していない^^f0^^9f

```

```

        ^^a6^^8a")
    print("[デバッグ]_music_genre_order_test_異常終了^^f0
        ^^9f^^a6^^8a")
    return -1
# あったらよい。pandas フレームにする
df = pd.read_csv(csv_path)

# 特徴量+one-hot ラベル作成
# 辞書型。cf は calculate_features の出力
row = cf

# 標準入力でlabelに値を入れる
input_labels = input("この曲を表す単語を\" \"なしスペース
    ( )区切りで入力しなさい^^f0^^9f^^a6^^8a:\n").split(" ")
#####for label in music_descriptive_labels:
#####if enable_override:
#####row[label] = 1 if label in input_labels else 0
#####else:
#####if label in input_labels:
#####row[label] = 1

###### 対象行を上書き
#####for col in row:
#####df.at[music_index, col] = row[col]
###### 保存
#####df.to_csv(csv_path, index=False)
#####print(f"[デバッグ]_csv{music_index+2}_行目に保存しました")

#_CTRL+_ENTERを押すだけで聞けるようにする
#_グローバル変数
listened_music_order_list = []
listening_music_genre = "classical"
#_save_feature_label_2_csv=False_はジャンル指定して、そのジャンルから
    まだ聞いている曲をランダムで流す
#_save_feature_label_2_csv=True_だとジャンル固定しない(入れても無視
    、聞いたリストも追加されない)。
#_その代わりにzcr を保存していないところを探す
def music_random_test(genre="classical",
    save_feature_label_2_csv=False, csv_path=csv_ssl_path,
    enable_override=True):

######_グローバル変数を関数内で書き込むために宣言
#####global listened_music_order_list, listening_music_genre

```

```

        if genre not in GENRE_LIST:
            print(f"[用法] {str(GENRE_LIST)} から選択しなさい^^f0^^9f^^a6^^8a")
            print("[デバッグ] music_random_test 異常終了^^f0^^9f^^a6^^8a")
            return -1

        # 現在聞いているジャンルの中のすでに聞いた曲数が、そのジャンルの全ての曲数に達したらリセット
        if len(listened_music_order_list) == 100 or (genre == "jazz" and len(listened_music_order_list) == 99):
            listened_music_order_list.clear()

        # 別ジャンルを聞くとときもリセット
        if genre != listening_music_genre:
            listened_music_order_list.clear()
            listening_music_genre = genre

        # 曲抽選機
        if save_feature_label_2_csv:
            # csv 読み込む
            df = pd.read_csv(csv_path)
            # zcr が 0.0 じゃないところ (無記入) を探す
            music_index = np.random.randint(0, 999)
            while df.at[music_index, 'zcr'] != 0.0:
                music_index = np.random.randint(0, 999)
            print(df.at[music_index, 'zcr'], music_index) # 後で消す
            g, o = music_index_2_genre_order(music_index)
            # index->genre, order->index という無駄なことをしている
            # が、面倒臭かったのでよし
            music_genre_order_test(genre=g, order=o,
                                   save_feature_label_2_csv=save_feature_label_2_csv,
                                   csv_path=csv_path,
                                   csv_ssl_path=csv_ssl_path,
                                   enable_override=enable_override)
        else:
            music_order = np.random.randint(1, 100)
            music_order_max_plus_one = 101
            if listening_music_genre == "jazz": # ジャズだけ一曲少ないからね
                music_order_max_plus_one = 100
            # 聞いているリストから探すよ
            music_order = np.random.randint(1,
                                             music_order_max_plus_one)

```

```

while music_order in listened_music_order_list:
    music_order = np.random.randint(1,
        music_order_max_plus_one)
    # そのジャンルのまだ聞いていない音楽の番号を聞いたリストに保存
    する
    listened_music_order_list.append(music_order)
    music_genre_order_test(genre=listening_music_genre,
        order=music_order, save_feature_label_2_csv=
        save_feature_label_2_csv, csv_path=csv_ssl_path,
        enable_override=enable_override)
    print(listened_music_order_list)

```

B CSV形式データセットの定義

ラベル付きデータおよびラベルなしデータを同一 CSV ファイルから条件分岐により取得するため, PyTorch の Dataset クラスを用いて専用のデータセットクラスを実装した.

ソースコード 3: CSVDataset クラス

```

# CSVを読む Dataset
class CSVDataset(Dataset):
    def __init__(self, csv_path, labeled=True):
        if not os.path.exists(csv_path):
            print("[エラー] csv ファイルが存在していない^f0^9f^a6
                ^8a")
            print("[デバッグ] CSVDataset 異常終了^f0^9f^a6^8a")
            return

        # csv 読み込み
        df = pd.read_csv(csv_path)
        features = df.iloc[:, :len(music_descriptive_features)].
            values
        labels = df.iloc[:, len(music_descriptive_features):].
            values

        if labeled:
            # ラベル付きデータ(1つ以上ラベルが付いている)
            mask = labels.sum(axis=1) > 0
            self.features = torch.tensor(features[mask], dtype=
                torch.float32)

```



```

        self.labels = torch.tensor(labels[mask], dtype=torch.
            float32)
    else:
        # ラベルなし特微量ありデータ(すべて 0、あるいは無視)
        mask = ((labels.sum(axis=1) == 0) & (features.sum(
            axis=1) != 0)) # & 前後で ( ) で囲わないと常に False
            になる
        self.features = torch.tensor(features[mask], dtype=
            torch.float32)
        self.labels = None

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        if self.labels is not None:
            return self.features[idx], self.labels[idx]
        else:
            return self.features[idx]

```

C マルチラベル分類ニューラルネットワーク

本研究で用いたモデルは、音楽特微量を入力とし、複数の単語ラベルの出現確率を同時に推定するマルチラベル分類ニューラルネットワークである。

ソースコード 4: MultiLabelNN の定義

```

# 複数ラベルに対応させるためのニューラルネットワーク
class MultiLabelNN(nn.Module):
    def __init__(self, input_dim, output_dim, hidden_dim=8,
        p_dropout=0.3):
        super().__init__()
        self.fc = nn.Sequential(
            # 入力層->中間層
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim), # バッチ正規化は神
            # 0.64->0.16になった
            nn.Sigmoid(), # ReLU より
            Sigmoidの方がよいことが判った。ロス 3桁くらい違う
            nn.Dropout(p=p_dropout),

```

```

        # 中間層->出力層
        nn.Linear(hidden_dim, output_dim),
        nn.BatchNorm1d(output_dim),
        nn.Sigmoid() # 出力ごと独立な確率
    )

    def forward(self, x):
        return self.fc(x)

```

D 半教師あり学習の学習ループ

ラベル付きデータによる教師あり学習と、ラベルなしデータに対する擬似ラベル学習を同一エポック内で交互に実行した。

ソースコード 5: 半教師あり学習ループ

```

# Dataset と DataLoader
labeled_dataset = CSVDataset(csv_path=csv_ssl_path, labeled=True)
unlabeled_dataset = CSVDataset(csv_path=csv_ssl_path, labeled=False)
labeled_loader = DataLoader(labeled_dataset, batch_size=8, shuffle=True)
unlabeled_loader = DataLoader(unlabeled_dataset, batch_size=8, shuffle=True)

model = MultiLabelNN(
    input_dim=len(music_descriptive_features),
    output_dim=len(music_descriptive_labels),
    hidden_dim=16,
    p_dropout=0.2
)
model.train()
criterion = nn.BCELoss() # バイナリクロスエントロピー
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

x_epoch = []
y_loss = []

for epoch in range(1, 1001):
    total_loss = 0.0

```

```

# ラベル付きデータでの学習
for inputs, labels in labeled_loader:
    outputs = model(inputs)
    loss = criterion(outputs, labels)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    total_loss += loss.item()

# ラベルなしデータで Pseudo-Label 学習
for inputs in unlabeled_loader:
    with torch.no_grad():
        pseudo_outputs = model(inputs)
        pseudo_labels = (pseudo_outputs > 0.5).float()

    outputs = model(inputs)
    pseudo_loss = criterion(outputs, pseudo_labels)

    optimizer.zero_grad()
    pseudo_loss.backward()
    optimizer.step()

    total_loss += 0.5 * pseudo_loss.item() # 疑似ラベルの影響を
        低減する

if epoch % 100 == 0:
    avg_loss = total_loss / (len(labeled_loader) + len(
        unlabeled_loader))
    print(f'Epoch_{epoch}, Avg_Loss:_{avg_loss:.4f}')
    x_epoch.append(epoch)
    y_loss.append(avg_loss)

plt.xlim((0., max(x_epoch)))
plt.ylim((0., max(y_loss)))
plt.plot(x_epoch, y_loss, 'o')
plt.show()

torch.save(model.state_dict(), model_ssl_save_path)

```

E 推論および単語ラベル抽出処理

推論時には、各ラベルの出現確率を算出し、確率の高い順に並べ替えた上で、品詞ごとに最大数を制限して単語列を生成した。

ソースコード 6: 推論およびラベル抽出

```
def model_genre_order_test(model, genre="classical", order=80):
    music_index = genre_order_2_music_index(genre=genre, order=
        order)
    if music_index == -1:
        print("[デバッグ] model_test 異常終了^^f0^^9f^^a6^^8a")
        return

    gtzan_music_sound_by_index(music_index)
    model_test(model, waveforms[music_index][START_SAMPLE_INDEX:
        n_samples], 22050)

def model_test(model, waveform, sr=22050):

    cf = calculate_features(waveform, sr=sr)

    # 推論モード&勾配不要
    model.eval()
    with torch.no_grad():
        # 特徴ベクトル → バッチ
        input_features = torch.tensor(list(cf.values()), dtype=
            torch.float32)
        input_batch = input_features.unsqueeze(0) # テンソル配列形
            式 [1, [zcr, spec_centroid, rms,...]]
        # モデル出力(確率)
        pred_labels_probability = model(input_batch).squeeze(0) #
            テンソル配列形式 [num_labels]

    pred_words = []
    # for i in range(len(music_descriptive_labels)):
    # if pred_labels_probability[i] >= SIGMOID_THRESFOLD:
    # pred_words.append(music_descriptive_labels[i])

    # print(pred_labels_probability)
    probs_sorted, idx_sorted = pred_labels_probability.sort(
        descending=True)

    adjectives = []
```

```

norns = []
color, season = ["", ""]
print("===全ラベルの確率(高い順)===")
for idx, prob in zip(idx_sorted.tolist(), probs_sorted.tolist()):
    label = music_descriptive_labels[idx]
    if label in Adjectives and len(adjectives) < 2:
        adjectives.append(label)
        pred_words.append(label)
    if label in Norns and len(norns) < 2:
        norns.append(label)
        pred_words.append(label)
    if label in Colors and color not in Colors:
        color = label
        pred_words.append(color)
    if label in Seasons and season not in Seasons:
        season = label
        pred_words.append(season)
    if label not in Adjectives and label not in Norns and
       label not in Colors and label not in Seasons and label
       != "鳴り響くように":
        if prob >= SIGMOID_THRESFOLD:
            pred_words.append(label)
print(f"{label: <7}: {prob*100:6.2f}%")

print("")
print("===抽出されたワード===")
print(" ".join(pred_words))

```

F 外部音源を用いた評価例

GTZAN データセットに含まれない外部音源に対しても、提案手法が適用可能であることを確認するため、mp3 形式の楽曲を入力として推論を行った。

ソースコード 7: 音響特徴量抽出関数

```

# 読み込みファイル名
mp3_load_path = "/content/drive/MyDrive/卒研/外部音源/放課後の異変.mp3"
# 読み取る

```

```

audio = AudioSegment.from_file(mp3_load_path, "mp3")
audio = audio.set_channels(1).set_frame_rate(SAMPLING_RATE) # おまじない
audio_5sec = audio[14000:16000] # 秒数で調節できる
waveform = np.array(audio_5sec.get_array_of_samples()).astype(np.float32) / (2**15) # 正規化
waveform_sound(waveform, SAMPLING_RATE)
model_test(model, waveform, SAMPLING_RATE)

```

G 特徴抽出プログラム

ソースコード 8: 音響特徴量抽出関数

```

# 特徴抽出関数
# 入力 : wav ファイル波形、サンプリング周波数
# 出力 : 辞書{zcr, spec_centroid, rms, spec_bw, tempo,
          spec_flatness, spec_contrast...7次元, mfcc_mean...13次元,
          mfcc_std13次元}
def calculate_features(waveform, sr=22050):

    cf = {}

    # ゼロクロス率(変化の激しさ)
    cf['zcr'] = np.mean(librosa.feature.zero_crossing_rate(
        waveform))

    # スペクトル中心(明るさ)
    cf['spec_centroid'] = np.mean(librosa.feature.
        spectral_centroid(y=waveform, sr=sr))

    # 音量(RMS エネルギー)
    cf['rms'] = np.mean(librosa.feature.rms(y=waveform))

    # スペクトル幅 (リッチさ)
    cf['spec_bw'] = np.mean(librosa.feature.spectral_bandwidth(y=
        waveform, sr=sr))

    # テンポ
    tempo, _ = librosa.beat.beat_track(y=waveform, sr=sr)
    cf['tempo'] = tempo

```

```

# スペクトル平坦性
cf['spec_flatness'] = np.mean(librosa.feature.
    spectral_flatness(y=waveform))

# スペクトルコントラスト 7次元
for i, value in enumerate(np.mean(librosa.feature.
    spectral_contrast(y=waveform, sr=sr), axis=1)):
    cf[f"spec_contrast{i}"] = value

# mfccメル周波数ケプストラム係数 mean 平均 std 標準偏差
mfcc = librosa.feature.mfcc(y=waveform, sr=sr, n_mfcc=13)
mfcc_mean = np.mean(mfcc, axis=1)
mfcc_std = np.std(mfcc, axis=1)
for i, value in enumerate(mfcc_mean):
    cf[f"mfcc{i}_mean"] = value
for i, value in enumerate(mfcc_std):
    cf[f"mfcc{i}_std"] = value

return cf

# 特徴量を形容詞・名詞に変換する関数 (仮)
def describe_features(cf):

    desc = []

    # ゼロクロス率から疾走感
    if cf['zcr'] > ZERO_CROSSING_RATE_FAST_BORDER:
        desc.append("疾走感のある (" + str(cf['zcr']) + ")")
    elif cf['zcr'] < ZERO_CROSSING_RATE_RELAX_BORDER:
        desc.append("ゆったり (" + str(cf['zcr']) + ")")
    else:
        desc.append("平均的 (" + str(cf['zcr']) + ")")

    # スペクトル中心から音の明るさ
    if cf['spec_centroid'] > SPECTRAL_CENTROID_BRIGHT_BORDER:
        desc.append("明るい (" + str(cf['spec_centroid']) + ")")
    elif cf['spec_centroid'] < SPECTRAL_CENTROID_DARK_BORDER:
        desc.append("暗い (" + str(cf['spec_centroid']) + ")")
    elif cf['spec_centroid'] < (SPECTRAL_CENTROID_BRIGHT_BORDER +
        SPECTRAL_CENTROID_DARK_BORDER)/2:
        desc.append("少し暗い (" + str(cf['spec_centroid']) + ")")
    else:
        desc.append("少し明るい (" + str(cf['spec_centroid']) + ")")

```

```

# 音量からエネルギー感
if cf['rms'] > RMS_BORDER:
    desc.append("力強い (" + str(cf['rms']) + ")")
else:
    desc.append("控えめ (" + str(cf['rms']) + ")")

# スペクトル幅からの音の豪華さ
if cf['spec_bw'] > SPECTRAL_BANDWIDTH_RICH_BORDER:
    desc.append("豪華 (" + str(cf['spec_bw']) + ")")
elif cf['spec_bw'] < SPECTRAL_BANDWIDTH_SIMPLE_BORDER:
    desc.append("地味 (" + str(cf['spec_bw']) + ")")
elif cf['spec_bw'] < (SPECTRAL_BANDWIDTH_RICH_BORDER +
    SPECTRAL_BANDWIDTH_SIMPLE_BORDER)/2:
    desc.append("少し地味 (" + str(cf['spec_bw']) + ")")
else:
    desc.append("少し豪華 (" + str(cf['spec_bw']) + ")")

# テンポから曲の速さ
if cf['tempo'] > 140:
    desc.append("速い" + str(cf['tempo']))
elif cf['tempo'] < 80:
    desc.append("遅い" + str(cf['tempo']))
else:
    desc.append("等速" + str(cf['tempo']))

# スペクトル平坦性からノイズの影響
if cf['spec_flatness'] > 0.3:
    desc.append("ノイジーな (" + str(cf['spec_flatness']) + ")")
elif cf['spec_flatness'] < 0.15:
    desc.append("澄んだ (" + str(cf['spec_flatness']) + ")")
else:
    desc.append("少しノイズを含む (" + str(cf['spec_flatness']) + ")")

# 最後にまとめる
return ", ".join(desc)

# 入力 : csvパス
# 出力 : csvの features を全曲分埋める
def music_features_save(csv_path=csv_path):

    # 保存先ファイルの存在確認

```



```

if not os.path.exists(csv_path):
    print("[エラー] 保存先のフォルダが存在していない^^f0^^9f^^a6
          ^^8a")
    print("[デバッグ] music_genre_order_test 異常終了^^f0^^9f^^
          a6^^8a")
    return -1

# あったらよい。pandas フレームにする
df = pd.read_csv(csv_path)

# 特徴抽出する
for music_index in range(0, 999):
    cf = calculate_features(waveforms[music_index][
        START_SAMPLE_INDEX:n_samples], sr=22050)
    # 対象行を上書き
    for col in cf:
        df.at[music_index, col] = cf[col]

# 保存
df.to_csv(csv_path, index=False)
print(f"[デバッグ] csv{csv_path} 行目に保存しました")

```

H GTZAN データセットの読み込み方法

ソースコード 9: GTZAN データセット読み込み関数

```

# 各音声ファイルから最初の 5秒 (22050Hz × 5秒 = 110250サンプル) を
  取得
TARGET_LENGTH = 110250
SAMPLE_RATE = 22050

# 対象ジャンル
genres = ["blues", "classical", "country", "disco", "hiphop",
          "jazz", "metal", "pop", "reggae", "rock"]

# waveforms にすべての読み込んだ波形データを格納する
waveforms = []

for genre in genres:
    for i in range(100):

```

```
file_path = f"/content/drive/MyDrive/卒研/archive/Data/  
genres_original/{genre}/{genre}.{str(i).zfill(5)}.wav"  
try:  
    y, _ = librosa.load(file_path, sr=SAMPLE_RATE, mono=  
        True)  
    waveforms.append(y[:TARGET_LENGTH])  
except Exception as e:  
    print(f"スキップ:{file_path}-{e}")
```